

Lecture 4: 神经网络与反向传播

Neural Networks and Backpropagation | 结构化知识笔记

整理自视频字幕 PDF 与 lecture_4.pdf

目录

内容完整性检查	3
0. 本讲和上一讲的关系	3
一句话核心	3
结构化要点	3
知识点联系	4
1. 从线性分类器到神经网络	4
一句话核心	4
结构化要点	4
直觉理解 + 类比	5
关键公式	5
易混淆点 / 常见误区	5
2. 为什么一定要有非线性激活函数	5
一句话核心	5
结构化要点	6
直觉理解 + 类比	6
易混淆点 / 常见误区	6
3. 常见激活函数	6
一句话核心	6
结构化要点	6
直觉理解 + 类比	7
易混淆点 / 常见误区	8
4. 网络容量、隐藏层大小与正则化	8
一句话核心	8
结构化要点	8
与上一讲串联	8
直觉理解 + 类比	8
易混淆点 / 常见误区	9
5. 生物神经元类比：能帮助理解，但不要过度相信	9
一句话核心	9
结构化要点	9
易混淆点 / 常见误区	9

6. 前向传播：神经网络如何产生预测	9
一句话核心	9
结构化要点	10
关键公式	10
直觉理解 + 类比	10
7. 为什么需要计算图	10
一句话核心	10
结构化要点	10
直觉理解 + 类比	11
8. 反向传播的核心：链式法则	11
一句话核心	11
结构化要点	11
三个重要术语	12
直觉理解 + 类比	13
9. 常见 Gate 的梯度流模式	13
一句话核心	13
结构化要点	13
易混淆点 / 常见误区	14
10. Sigmoid 的反向传播	14
一句话核心	14
关键公式	15
直觉理解	15
易混淆点 / 常见误区	15
11. Forward / Backward API：深度学习框架怎么实现反传	15
一句话核心	15
结构化要点	15
与 PyTorch 的联系	16
易混淆点 / 常见误区	16
12. 标量、向量、矩阵中的反向传播	16
一句话核心	16
结构化要点	16
关键规律	17
直觉理解	18
13. ReLU 的向量反向传播	18
一句话核心	18
结构化要点	18
直觉理解	19
易混淆点 / 常见误区	19
14. 矩阵乘法的反向传播	19
一句话核心	19
结构化要点	19
形状检查	20

为什么不显式构造 Jacobian?	20
直觉理解	20
易混淆点 / 常见误区	21
15. 本讲与前面知识点的完整串联	21
一句话核心	21
串联图	21
本讲真正解决的问题	21
和下一讲的关系	22
16. 本讲最重要的复习抓手	22
必须会说清楚	22
17. 常见误区汇总	23
18. 自测小问题	23
一句话收束	23

内容完整性检查

本次材料包括三份视频课字幕 PDF 与一份课件 PDF。整体判断：**内容完整，可以作为一课整理。**

- lecture_4.pdf 覆盖 Lecture 4: Neural Networks and Backpropagation，从上一讲回顾、神经网络、激活函数、前向传播、计算图、反向传播，到向量/矩阵反传和下一讲 CNN 预告。
- 三份字幕 PDF 是连续衔接的：第 1 份从开头讲到神经网络规模、容量与正则化；第 2 份接着讲正则化、计算图与反向传播；第 3 份接着 sigmoid 局部梯度，继续讲梯度流模式、forward/backward API、向量/矩阵反传，并以“下一节进入卷积神经网络”结束。
- 字幕中有少量 ASR 识别错误，整理时已按上下文修正。例如：overfeeding 应为 **overfitting**，scholar 应为 **scalar**，paralyzing 应为 **penalizing**。

0. 本讲和上一讲的关系

一句话核心

上一讲解决“如何定义损失、如何优化参数”，这一讲解决“当模型变成多层神经网络时，梯度到底怎么高效算出来”。

结构化要点

前面课程主线是：

图像分类

- 线性分类器 $f(x, W) = Wx + b$
- 损失函数：Softmax / Hinge
- 正则化：防止过拟合
- 优化：用梯度下降、SGD、Adam 找更好的 W
- 本讲：神经网络 + 反向传播，解决复杂模型的梯度计算

上一讲我们已经知道，训练模型本质上要计算：

$$\nabla_W L$$

也就是损失函数 L 对参数 W 的梯度。问题是：线性模型时还能手算，一旦模型变成几十层、上百层，手推导就非常痛苦。**反向传播 (Backpropagation)** 就是为了解决这个问题的。

知识点联系

上一讲的核心更新公式是：

$$W \leftarrow W - \alpha \nabla_W L$$

这一讲要回答的是：如果 L 来自一个复杂神经网络， $\nabla_W L$ 到底怎么得到？

1. 从线性分类器到神经网络

一句话核心

神经网络是在多个线性变换之间插入非线性激活函数，从而获得比线性分类器更强的表达能力。

结构化要点

之前的线性分类器是：

$$f = Wx$$

它只做一次线性变换。

两层神经网络可以写作：

$$f = W_2 \max(0, W_1 x)$$

其中：

- $x \in \mathbb{R}^D$ ：输入向量。
- $W_1 \in \mathbb{R}^{H \times D}$ ：第一层权重。
- H ：隐藏层神经元个数。
- $W_2 \in \mathbb{R}^{C \times H}$ ：第二层权重。
- C ：类别数。
- $\max(0, \cdot)$ ：激活函数，这里是 ReLU。

三层神经网络可以写成：

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

课件中用 $3072 \rightarrow 100 \rightarrow 10$ 的例子说明：输入图像先被映射到 100 个隐藏单元，再输出 10 个类别分数。

直觉理解 + 类比

线性分类器像是直接拿整张图片和每个类别模板比较：

这张图像像不像猫？像不像车？像不像鸟？

神经网络则多了一层中间表示：第一层可能学到“边缘、眼睛、纹理、局部形状”等中间特征，第二层再把这些特征组合成类别判断。

线性分类器：直接判断“这是不是猫”

神经网络：先找“耳朵、眼睛、毛发、轮廓”，再综合判断“这是不是猫”

这就是神经网络的层级计算（Hierarchical Computation）。

关键公式

两层神经网络：

$$h = \max(0, W_1 x)$$

$$f = W_2 h$$

合起来：

$$f = W_2 \max(0, W_1 x)$$

其中 h 是隐藏层表示（Hidden Representation）。

易混淆点 / 常见误区

“两层神经网络”通常指有两层带参数的层。输入层不算参数层，所以：

输入层 → 隐藏层 → 输出层

常被称为：

- 2-layer neural network
- 1-hidden-layer neural network

2. 为什么一定要有非线性激活函数

一句话核心

没有非线性激活函数，多层线性变换叠起来仍然只是一个线性变换。

结构化要点

如果去掉 ReLU：

$$f = W_2 W_1 x$$

令：

$$W_3 = W_2 W_1$$

那么：

$$f = W_3 x$$

这又变回了一个线性分类器。

所以，神经网络真正变强的关键不是“矩阵乘法变多了”，而是中间插入了非线性函数。

直觉理解 + 类比

只叠线性层就像连续做几次“拉伸、旋转、平移”，最终仍然只是一次大的线性变换。

加入非线性激活，相当于允许模型对空间进行“折叠、弯曲、切分”，这样原本一条直线分不开的数据，就可能在新空间中变得可分。

易混淆点 / 常见误区

不要把“层数多”本身理解成神经网络强大的根本原因。真正关键是：

线性变换 + 非线性激活 + 多层组合

如果没有非线性，再多层也只是线性模型。

3. 常见激活函数

一句话核心

激活函数（Activation Function）负责引入非线性，是神经网络表达复杂函数的关键部件。

结构化要点

本讲提到的激活函数包括：

3.1 ReLU

$$\text{ReLU}(x) = \max(0, x)$$

特点：

- 简单。

- 计算快。
- 通常是默认选择。
- 正数直接通过，负数变成 0。

3.2 Leaky ReLU

$$\text{LeakyReLU}(x) = \max(0.1x, x)$$

作用：缓解 ReLU 的“死亡神经元”问题。

3.3 ELU

指数线性单元（Exponential Linear Unit），负半轴更平滑，可能帮助优化。

3.4 GELU

高斯误差线性单元（Gaussian Error Linear Unit），常见于 Transformer。

3.5 SiLU / Swish

Sigmoid Linear Unit，常见于一些现代 CNN 架构，例如 EfficientNet。

3.6 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

输出范围：

$$(0, 1)$$

3.7 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

输出范围：

$$(-1, 1)$$

直觉理解 + 类比

激活函数像神经网络里的“开关”或“门”。不是所有信号都原样通过，而是根据输入强弱决定输出。

ReLU 很像一个单向阀：

输入 > 0 ：放行
输入 ≤ 0 ：截断

易混淆点 / 常见误区

Sigmoid 和 Tanh 以前很常用，但现在很少作为深层网络中间层的默认激活函数，因为它们会把输入压缩到很窄的范围，容易导致**梯度消失 (Vanishing Gradient)**。

但它们仍可能用于输出层，例如二分类输出概率时常用 sigmoid。

4. 网络容量、隐藏层大小与正则化

一句话核心

隐藏层神经元越多，模型容量越大；但容量过大容易过拟合，因此通常用正则化控制，而不是简单把网络做小。

结构化要点

隐藏神经元数量变化会影响决策边界复杂度：

3 hidden neurons：边界简单，可能欠拟合

6 hidden neurons：边界更灵活

20 hidden neurons：边界复杂，可能过拟合

更多神经元意味着更高容量 (Capacity)。容量高的模型可以拟合更复杂的函数，但也更容易记住训练数据中的噪声。

课程中特别强调：通常不要把“网络大小”直接当作 regularizer，而是选择稍微足够大的网络，再通过正则化强度 λ 来控制过拟合。

与上一讲串联

上一讲讲过总损失：

$$L(W) = \frac{1}{N} \sum_i L_i + \lambda R(W)$$

这一讲把它放进神经网络中：

- L_i ：模型预测和真实标签之间的差距。
- $R(W)$ ：限制权重，防止过拟合。
- λ ：正则化强度。

当 λ 太大时，模型被限制太强，容易欠拟合；当 λ 太小时，模型可能过拟合。

直觉理解 + 类比

网络容量像笔的精细程度：

- 太粗的笔画不出复杂边界。
- 太细的笔可以画出训练点的每一个弯弯绕绕。
- 但考试看的是新数据，不是把训练集描边描得多精细。

所以常见策略是：

给模型足够表达能力
再用正则化控制它不要乱发挥

易混淆点 / 常见误区

“更大的网络一定更好”是错的。更大的网络只是有更强的表达能力，是否泛化更好，还取决于数据量、正则化、优化器、学习率、训练策略等。

5. 生物神经元类比：能帮助理解，但不要过度相信

一句话核心

人工神经网络受到生物神经元启发，但它不是大脑的精确模拟。

结构化要点

课程中提到生物神经元大致包括：

- 树突 (Dendrite)：接收信号。
- 细胞体 (Cell Body)：整合信号。
- 轴突 (Axon)：传递信号。
- 突触末端 (Presynaptic Terminal)：连接其他神经元。

人工神经元中：

$$h = \sigma(Wx + b)$$

可以粗略理解为：

输入信号 x

→ 权重 W 调整不同输入的重要性

→ 加权求和 + bias

→ 激活函数决定输出

但课程也强调：生物神经元复杂得多，人工神经网络为了计算效率通常组织成规则层结构，不能把“神经网络像大脑”这个类比当成严格事实。

易混淆点 / 常见误区

“神经网络”这个名字容易让人误以为它在真实模拟大脑。更准确地说，本课讨论的是：

- 全连接网络 (Fully Connected Network)
- 多层感知机 (Multilayer Perceptron, MLP)

它们是数学模型，不是完整脑模型。

6. 前向传播：神经网络如何产生预测

一句话核心

前向传播 (Forward Pass) 就是从输入开始，一层一层计算，最终得到预测和损失。

结构化要点

一个简单两层神经网络训练过程包括：

1. 定义网络结构
2. 初始化参数 W_1, W_2
3. 前向传播，得到预测 y_{pred}
4. 计算 loss
5. 反向传播，计算梯度
6. 梯度下降，更新 W_1, W_2

关键公式

两层网络前向传播：

$$h = \sigma(XW_1)$$

$$\hat{y} = hW_2$$

其中：

- X ：输入数据。
- W_1 ：输入层到隐藏层的权重。
- h ：隐藏层激活。
- W_2 ：隐藏层到输出层的权重。
- $\hat{y}$ ：模型预测。

直觉理解 + 类比

前向传播像流水线生产：

原材料输入

- 第一道工序提取初级特征
- 第二道工序组合特征
- 最后得到产品，也就是预测结果

反向传播则像质检反馈：发现最终产品有误后，沿流水线反向追责，判断每个工序该怎么调整。

7. 为什么需要计算图

一句话核心

计算图（Computational Graph）把复杂函数拆成简单操作节点，让反向传播可以局部、模块化地计算梯度。

结构化要点

完整神经网络可能包含：

- 矩阵乘法。

- 加法。
- 激活函数。
- 损失函数。
- 正则化项。
- 更复杂的模块，例如卷积、循环结构、注意力等。

如果每次都手推整个模型的导数，会遇到三个问题：

1. 矩阵微积分很繁琐。
2. 换一个 loss 就要重新推导。
3. 模型复杂后几乎不可行。

所以更好的办法是：

把整个模型拆成计算图

每个节点只负责自己的 forward 和 backward

全局梯度通过链式法则自动传回去

直觉理解 + 类比

计算图像一张“加工流程图”。每个节点只知道：

我收到什么输入

我做了什么运算

我输出什么

如果后面告诉我输出错了，我该把责任分给谁

这样就不用每次从头推导整条生产线。

8. 反向传播的核心：链式法则

一句话核心

反向传播就是沿计算图反向递归应用链式法则，把最终 loss 对每个变量的影响算出来。

结构化要点

课程用简单函数举例：

$$f(x, y, z) = (x + y)z$$

令：

$$q = x + y$$

则：

$$f = qz$$

前向传播：

$x, y \rightarrow q = x + y$

$q, z \rightarrow f = qz$

反向传播从最终输出 f 开始：

$$\frac{\partial f}{\partial f} = 1$$

然后逐步往前传：

$$\frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial q} = z$$

对于 x, y ，需要链式法则：

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

三个重要术语

8.1 上游梯度 (Upstream Gradient)

从后面节点传来的梯度。

例如：

$$\frac{\partial L}{\partial z}$$

表示最终损失 L 对当前节点输出 z 的梯度。

8.2 局部梯度 (Local Gradient)

当前节点自己的导数。

如果：

$$z = f(x, y)$$

那么局部梯度是：

$$\frac{\partial z}{\partial x}, \quad \frac{\partial z}{\partial y}$$

8.3 下游梯度 (Downstream Gradient)

当前节点传给前面节点的梯度。

基本规律：

$$\text{下游梯度} = \text{上游梯度} \times \text{局部梯度}$$

直觉理解 + 类比

反向传播像事故追责：

最终 loss 变大了

→ 问输出层：你对错误贡献多少？

→ 输出层再问上一层：你的输入对我的错误贡献多少？

→ 一层层往前传，最后知道每个参数该怎么改

这不是“神经网络自己神奇学会”，而是非常机械地重复链式法则。

9. 常见 Gate 的梯度流模式

一句话核心

反向传播中，一些基本运算的梯度模式可以直接记住：加法分发、乘法交换、复制相加、最大值路由。

结构化要点

9.1 Add Gate：梯度分发器

前向：

$$z = x + y$$

局部梯度：

$$\frac{\partial z}{\partial x} = 1, \quad \frac{\partial z}{\partial y} = 1$$

所以反向时，上游梯度原样分给两个输入。

加法 forward：两个数相加

加法 backward：梯度原样分发

9.2 Multiply Gate: 交换乘法器

前向:

$$z = xy$$

局部梯度:

$$\frac{\partial z}{\partial x} = y, \quad \frac{\partial z}{\partial y} = x$$

所以反向时, 传给 x 的梯度乘以 y , 传给 y 的梯度乘以 x 。

乘法 backward: 拿“另一个输入”来乘

9.3 Copy Gate: 梯度相加器

如果一个变量被复制到多个分支, 那么反向传播时, 这些分支传回来的梯度要相加。

forward: 一个变量分叉给多个地方用

backward: 多个地方的梯度加回同一个变量

9.4 Max Gate: 梯度路由器

前向:

$$z = \max(x, y)$$

反向时, 梯度只传给前向传播中取到最大值的那个输入。

谁赢了 $\max$, 梯度就走向谁

这也是 ReLU 反向传播的基础:

$$\text{ReLU}(x) = \max(0, x)$$

当 $x > 0$, 梯度通过; 当 $x \leq 0$, 梯度为 0。

易混淆点 / 常见误区

Max Gate 不是把梯度平均分给所有输入。它只传给前向传播时被选中的最大值路径。

10. Sigmoid 的反向传播

一句话核心

Sigmoid 的导数可以用 sigmoid 自己表示, 因此在计算图中很方便。

关键公式

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

它的导数:

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

直觉理解

Sigmoid 在中间变化最快，在两端趋于平坦。所以：

- 输入接近 0 时，梯度较大。
- 输入很大或很小时，梯度接近 0。

这也是为什么 sigmoid 容易导致梯度消失。

易混淆点 / 常见误区

Sigmoid 的导数不是：

$$1 - \sigma(x)$$

而是：

$$\sigma(x)(1 - \sigma(x))$$

少乘一个 $\sigma(x)$ 是常见错误。

11. Forward / Backward API：深度学习框架怎么实现反传

一句话核心

现代框架把每个操作封装成 forward 和 backward：forward 算输出并缓存必要值，backward 用链式法则算梯度。

结构化要点

每个节点 / 操作通常做两件事。

Forward

输入 → 计算输出

保存 backward 需要的中间值

Backward

接收 upstream gradient
乘以 local gradient
输出 downstream gradient

例如 Multiply Gate:

前向:

$$z = xy$$

要保存 x, y , 因为反向时需要:

$$\frac{\partial z}{\partial x} = y, \quad \frac{\partial z}{\partial y} = x$$

与 PyTorch 的联系

你平时写 PyTorch:

```
loss.backward()
```

它背后做的就是:

沿着计算图从 loss 往回走
每个节点调用自己的 backward
把梯度逐层传回参数

易混淆点 / 常见误区

backward() 不是重新跑一遍 forward。forward 阶段已经保存了一些中间结果, backward 是利用这些缓存结果计算梯度。

12. 标量、向量、矩阵中的反向传播

一句话核心

无论变量是标量、向量、矩阵还是张量, 变量对 loss 的梯度形状总是和变量本身相同。

结构化要点

课程从标量扩展到向量和矩阵。

12.1 标量到标量

$$y = f(x)$$

导数是一个标量:

$$\frac{dy}{dx}$$

12.2 向量到标量

$$y = f(x), \quad x \in \mathbb{R}^n, y \in \mathbb{R}$$

导数是梯度向量：

$$\nabla_x y$$

它表示：每个 x_i 改变一点， y 会变多少。

12.3 向量到向量

$$y = f(x), \quad x \in \mathbb{R}^n, y \in \mathbb{R}^m$$

导数是雅可比矩阵（Jacobian）：

$$J_{ij} = \frac{\partial y_i}{\partial x_j}$$

它表示：每个输入元素如何影响每个输出元素。

关键规律

如果：

$$x \in \mathbb{R}^D$$

那么：

$$\frac{\partial L}{\partial x} \in \mathbb{R}^D$$

如果：

$$W \in \mathbb{R}^{D \times M}$$

那么：

$$\frac{\partial L}{\partial W} \in \mathbb{R}^{D \times M}$$

这句话特别重要：

对某个变量求 loss 的梯度，梯度形状一定和这个变量一样。

直觉理解

梯度告诉你“这个变量的每个位置应该怎么改”。如果变量有 100 个元素，当然需要 100 个梯度值；如果变量是一个矩阵，每个矩阵元素都需要一个对应的梯度。

13. ReLU 的向量反向传播

一句话核心

ReLU 是逐元素操作，所以它的 Jacobian 很稀疏，实际实现时不显式构造 Jacobian，只做逐元素 mask。

结构化要点

ReLU：

$$z = \max(0, x)$$

对于每个元素：

$$\frac{\partial z_i}{\partial x_i} = \begin{cases} 1, & x_i > 0 \\ 0, & x_i \leq 0 \end{cases}$$

如果：

$$x = [1, -2, 3, -1]$$

则：

$$z = [1, 0, 3, 0]$$

假设上游梯度：

$$\frac{\partial L}{\partial z} = [4, -1, 5, 9]$$

那么下游梯度：

$$\frac{\partial L}{\partial x} = [4, 0, 5, 0]$$

因为只有 $x > 0$ 的位置允许梯度通过。

直觉理解

ReLU 前向传播时像筛子：

正数留下
负数清零

反向传播时也一样：

前向留下的位置，梯度通过
前向清零的位置，梯度也被截断

易混淆点 / 常见误区

ReLU 的梯度不是对所有正向输出都传 1，而是把上游梯度乘以 0/1 mask：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \odot \mathbf{1}_{x>0}$$

其中 $\odot$ 表示逐元素乘法。

14. 矩阵乘法的反向传播

一句话核心

矩阵乘法反向传播不用显式构造巨大 Jacobian，而是用矩阵乘法公式直接算梯度。

结构化要点

设：

$$Y = XW$$

其中：

- $X \in \mathbb{R}^{N \times D}$
- $W \in \mathbb{R}^{D \times M}$
- $Y \in \mathbb{R}^{N \times M}$

给定上游梯度：

$$\frac{\partial L}{\partial Y} \in \mathbb{R}^{N \times M}$$

则：

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^T$$

$$\frac{\partial L}{\partial W} = X^T \frac{\partial L}{\partial Y}$$

形状检查

$$\frac{\partial L}{\partial Y} W^T$$

形状是：

$$(N \times M)(M \times D) = N \times D$$

正好和 X 一样。

$$X^T \frac{\partial L}{\partial Y}$$

形状是：

$$(D \times N)(N \times M) = D \times M$$

正好和 W 一样。

为什么不显式构造 Jacobian?

如果 batch size $N = 64$ ，维度 $D = M = 4096$ ，矩阵乘法节点的 Jacobian 可能达到约 256GB。实际中必须隐式计算，不会显式存这个巨大矩阵。

直觉理解

矩阵乘法反向传播其实是乘法 gate 的高维版本。

标量乘法：

$$z = xy$$

反向时：

给 x 的梯度要乘 y

给 y 的梯度要乘 x

矩阵乘法也是类似：

给 X 的梯度，要用 W

给 W 的梯度，要用 X

只是为了形状匹配，需要转置。

易混淆点 / 常见误区

很多初学者会背公式，但忘了形状。最稳的方法是用形状检查推公式：

dX 形状必须等于 X

dW 形状必须等于 W

所以：

$$dX = dYW^T$$

$$dW = X^T dY$$

15. 本讲与前面知识点的完整串联

一句话核心

前几讲搭好了“训练机器学习模型”的框架，本讲把模型从线性分类器升级为神经网络，并用反向传播补上复杂模型训练的关键环节。

串联图

Lecture 1 / 2：图像分类与线性分类器

输入图像 $x \rightarrow$ 线性打分 $f = Wx \rightarrow$ 输出类别分数 s

Lecture 3：损失、正则化与优化

类别分数 $s \rightarrow$ loss L

总损失 = data loss + regularization

用梯度下降 / SGD / Adam 更新 W

Lecture 4：神经网络与反向传播

线性打分升级为多层网络：

$f = W_2 \text{ReLU}(W_1 x)$

问题：复杂模型中 $\partial L / \partial W$ 怎么算？

答案：计算图 + 反向传播

本讲真正解决的问题

上一讲我们说：

$$W \leftarrow W - \alpha \nabla_W L$$

但当模型是：

$$f = W_3 \sigma(W_2 \sigma(W_1 x))$$

甚至更复杂的 CNN、Transformer 时， $\nabla_W L$ 不可能每次手推。

所以本讲给出的核心工具是：

计算图负责记录模型怎么从 x 算到 L
反向传播负责从 L 反向算出所有参数的梯度
优化器负责用这些梯度更新参数

和下一讲的关系

下一讲进入卷积神经网络 (Convolutional Neural Networks, CNN)。CNN 本质上也还是：

一堆操作节点组成的计算图

forward 得到 loss

backward 得到梯度

optimizer 更新参数

所以本讲是后面 CNN、RNN、Transformer 的底层基础。

16. 本讲最重要的复习抓手

必须会说清楚

1. **为什么线性分类器不够？**
因为它只能形成线性决策边界，很多数据非线性可分。
2. **神经网络比线性分类器强在哪里？**
它通过多层线性变换 + 非线性激活函数，学习复杂特征变换。
3. **为什么没有激活函数就不行？**
多个线性变换相乘仍是线性变换。
4. **ReLU 做什么？**
正数通过，负数清零，引入非线性。
5. **反向传播本质是什么？**
沿计算图反向递归使用链式法则。
6. **upstream / local / downstream gradient 是什么？**
上游梯度来自后面，局部梯度来自当前节点，下游梯度传给前面。
7. **为什么要计算图？**
为了模块化地计算复杂模型的梯度，避免手推整个模型。
8. **为什么不显式构造 Jacobian？**
太大，内存不可接受；实际用隐式矩阵运算。
9. **矩阵乘法反传公式是什么？**

$$Y = XW$$

$$dX = dY W^T$$

$$dW = X^T dY$$

10. 为什么梯度形状和变量形状一样？

因为每个变量元素都需要一个对应的“该怎么改”的信号。

17. 常见误区汇总

1. 神经网络只是很多线性分类器叠起来。

少了激活函数就确实只是线性分类器；关键是非线性。

2. ReLU 的作用只是把负数变 0。

更重要的是它引入非线性，让模型能表达复杂函数。

3. 网络越大越好。

网络越大容量越强，但也更容易过拟合，需要正则化控制。

4. 反向传播是一个新的数学规则。

它不是新规则，本质就是链式法则的系统化、模块化应用。

5. 每个节点都要知道整个网络。

不需要。每个节点只需要知道自己的 local gradient 和传来的 upstream gradient。

6. 向量/矩阵反传要真的构造 Jacobian。

理论上可以这么想，但实现中绝不会显式构造巨大 Jacobian。

7. `loss.backward()` 很神秘。

它就是从 loss 出发，沿计算图调用每个节点的 backward。

18. 自测小问题

1. 为什么 $W_2 W_1 x$ 仍然只是线性分类器？

2. ReLU 的 forward 和 backward 分别做了什么？

3. 如果一个变量在 forward 中被多个分支使用，backward 时梯度应该怎么处理？

4. 对于 $f = (x + y)z$ ，为什么 $\frac{\partial f}{\partial x} = z$ ？

5. 如果 $Y = XW$ ，为什么 $dW = X^T dY$ ？

6. 为什么我们说“梯度形状一定和变量形状相同”？

7. 为什么现代深度学习框架必须保存 forward 中的一些中间值？

一句话收束

神经网络负责把线性分类器升级成强表达模型，反向传播负责把复杂模型里的每个参数该怎么改算出来。